

MyPlace

Crashing Out

Ali Abdulrazzak, Alex DiMichele, Chris Ross, Sebastian Tran, Logan Yates

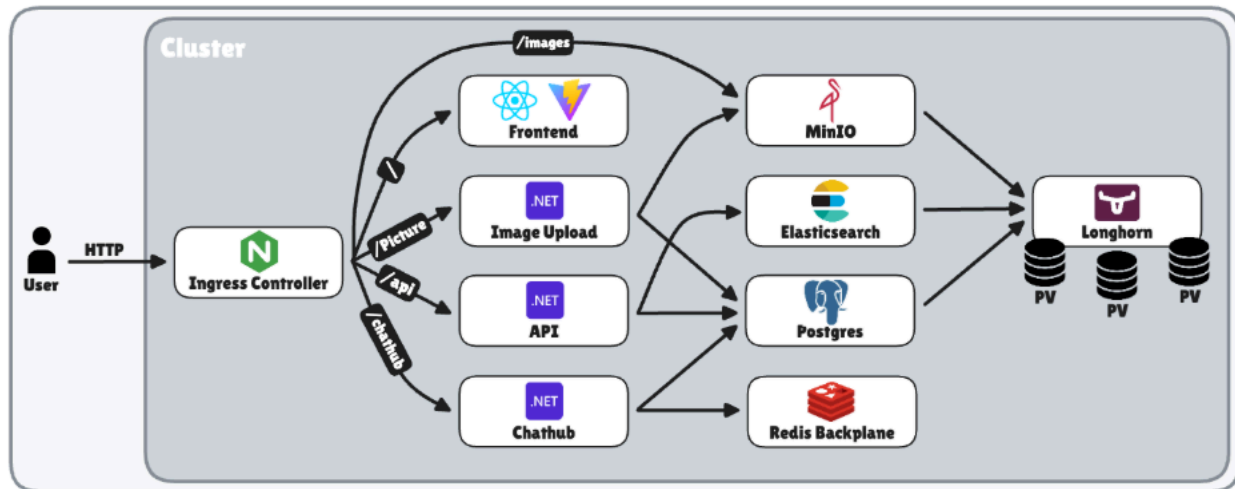
Overview

Roommate finding can be an egregious process.

Different living styles, tolerances for cleanliness, and acceptable noise levels are all factors that can lead to conflict among roommates.

The main goal of our app, MyPlace, is to streamline the process of finding roommates by providing an easy to use recommendation system and messaging platform. By providing a recommendation system that puts living style needs first, we hope to make finding a roommate a much less stressful endeavor.

Architecture



Nginx Ingress Controller:

The Nginx Ingress Controller will be responsible for routing traffic to the correct deployment. By including an ingress, each service associated with a deployment can be a simple ClusterIP as well. The reason for including an ingress will be further elaborated on in chapter 2, but briefly, websocket traffic requires an ingress.

Frontend

Our frontend will be served via Nginx. This will be the endpoint for our frontend interface.

API

The API will be responsible for creating profiles/logging in, keeping track of matches, and providing recommendations.

ChatHub

Our ChatHub will be responsible for handling the websocket connections to enable messaging in our application.

Redis Backplane

The Redis backplane will allow for sharing connection information between ChatHub pods. This enables sending messages between what would have been independent pods. In addition, we can also use redis as a key-value storage for storing routing information.

Image Upload Service

Images will be an important part of our app, as all profiles will require images that other users will view. The image upload service will handle this operation.

Postgres / CloudnativePG

Postgresql will be where all profiles, messages, and picture locations will be stored.

ElasticSearch

ElasticSearch will be an interface for our API to provide curated roommate recommendations.

MINIO

Provides object storage for files. Abstracts the file system to make image handling much easier. Also provides an endpoint that the frontend can directly access for images, instead of going through API.

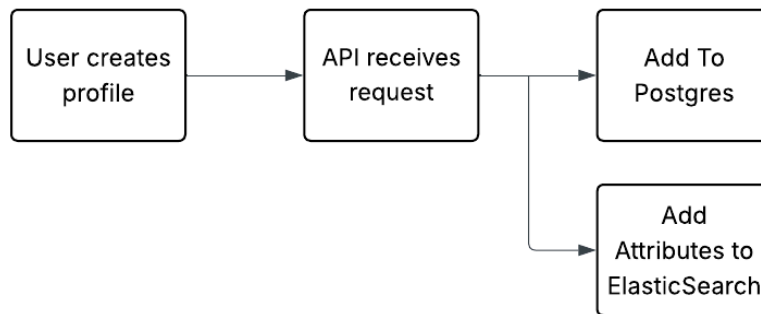
Longhorn

Handles persistent volume management for bare metal hardware. Longhorn provides distributed storage across nodes and resiliency for failover. Our databases will connect to Longhorn to get storage persistence without much configuration.

Data Flow

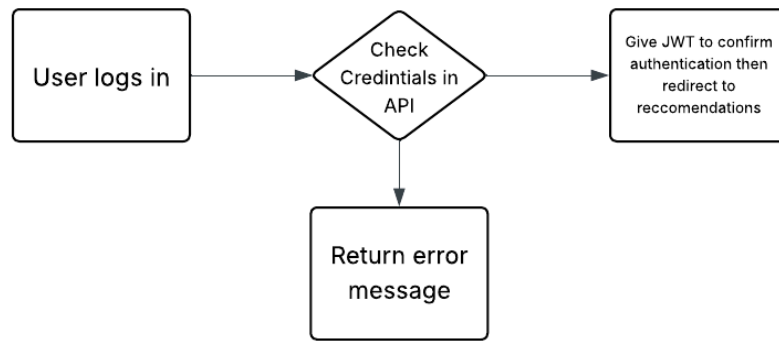
Registering a User:

As a user, I want to be able to register an account so I can create a profile.



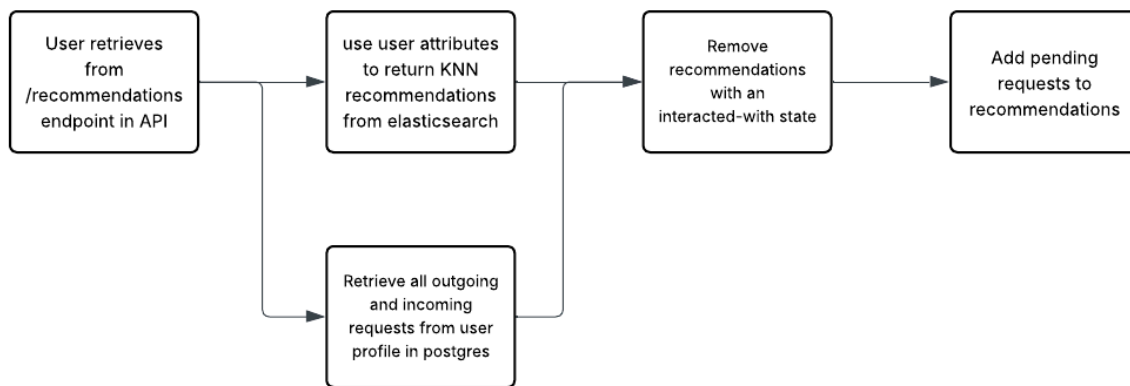
Logging In:

As a user, I want to securely log into my account so I can access my profile and matches.

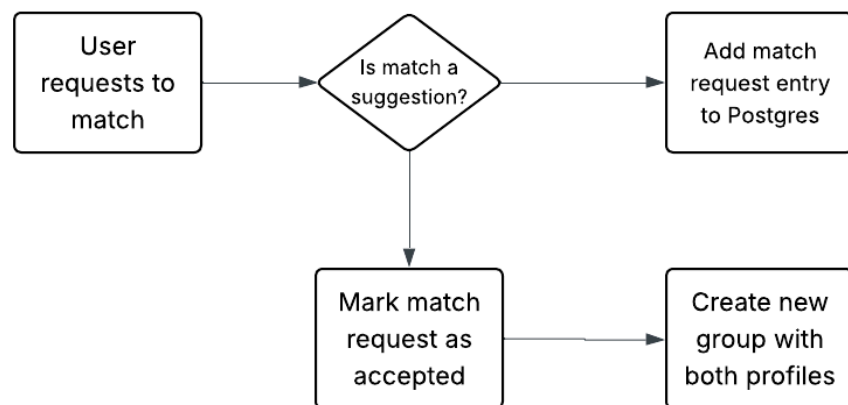


Finding Matches / Recommendations:

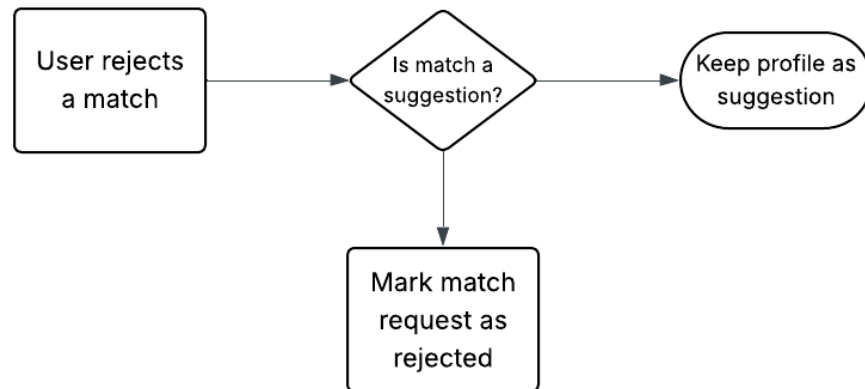
As a user, I want to view suggested roommates so I can match with them.



As a user, I would like to send a request to a potential roommate

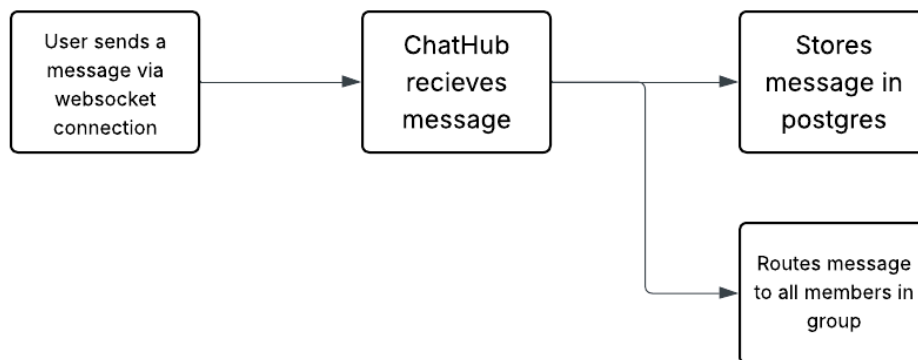


As user, I would like to reject a roommate request



Messaging:

As a user, I want to be able to message a match so I can determine if we're actually compatible.



As a user, I would like to receive a message from another user in real time

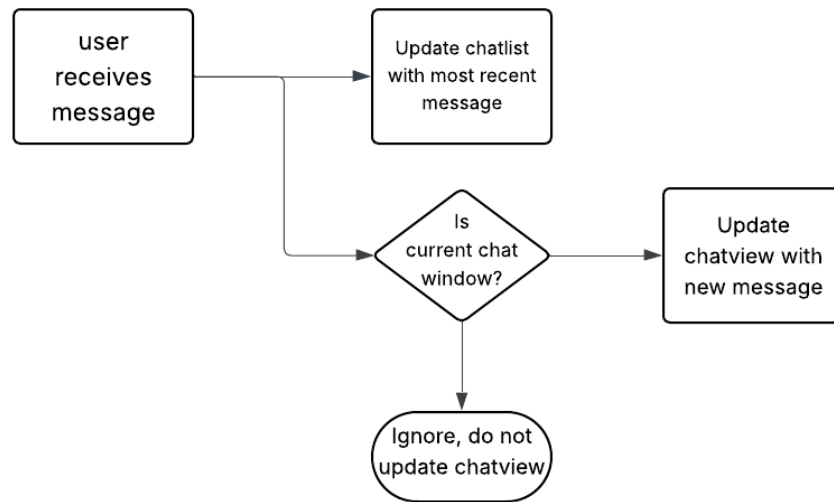
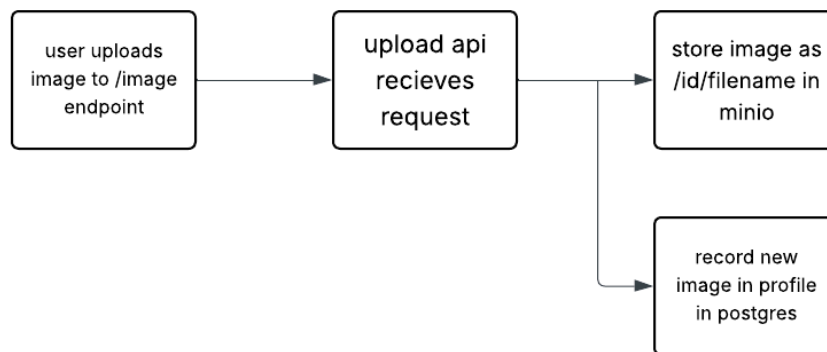


Image Upload

As a user, I would like to upload an image to share what I look like.



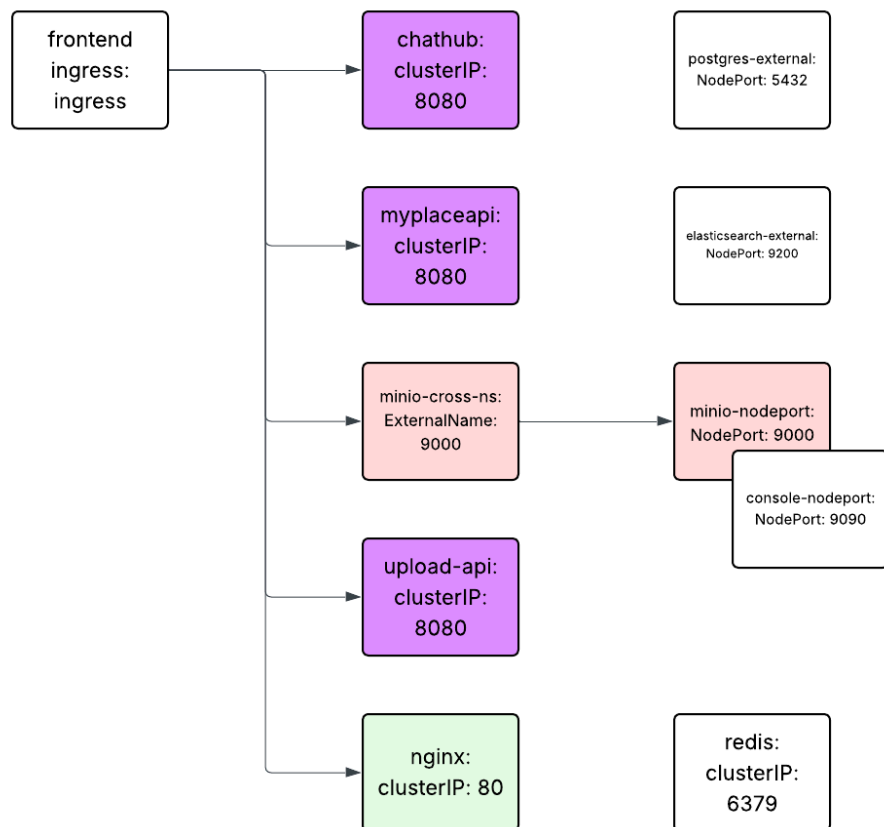
Pod Groupings + Side Cars

Deployment * replicas / Cluster	Pods
Deployment: Myplaceapi * 2	Initcontainer: migration-application Containers: stran9682/myplaceapi
Cluster: Elasticsearch (v. 9.2.0)	Nodeset: 1x Master (cluster management) 2x Data (data and indexing)

Tenant: Minio (v. 04.08)	Pool: 4x Nodes
Cluster: Postgres (v. 16.2)	3x Instances
Deployment : Chathub * 2	Containers: stran9682/Chathub
Deployment: Nginx * 1	Containers: stran9682/myplace-frontend
Deployment: Redis * 1	Containers: redis
Deployment: Upload-api	Containers: stran9682/uploadapi

Myplaceapi, Chathub, Upload API, and MigrationApplication use the official .net 8.0 image as a base image with a multi stage build to build their associated project into the image.

Services/Routing



Colored services are accessible to the user via the ingress, but an oddity being the minio-nodeport service. The minio-nodeport service is in a different namespace than all the other services, which required the minio-cross-ns ExternalName service to forward traffic to.

ClusterIP is acceptable for many of the services as they are exposed via the ingress. We have configured the databases to use nodeport, but they can also use ClusterIP as well, as there shouldn't be a need to expose them to the outside world.

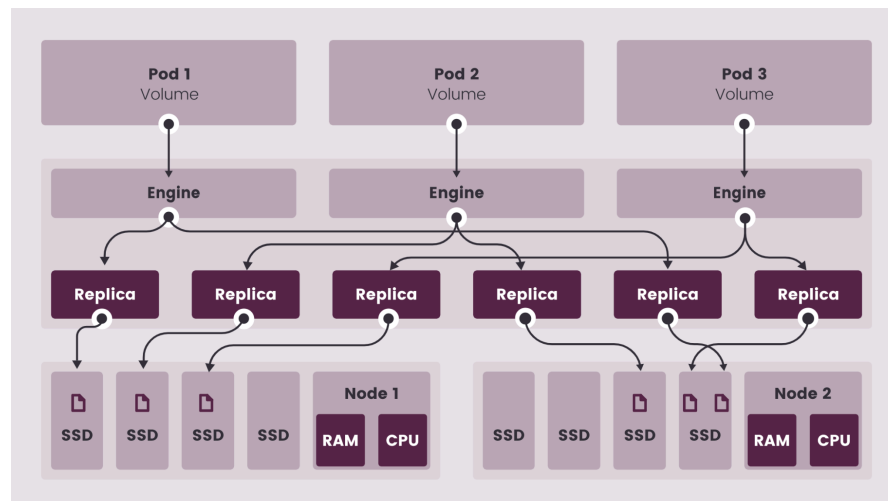
Scaling/Self Healing Results

Each of the .NET pods can scale infinitely, as they exist independent of each other. To demonstrate scaling we set all of them to 2, as shown in the pod groupings / sidecar section.

Self healing was shown when we updated the image to push a change to the pods. We deleted the pods, and the deployment, wanting to keep 2 pods at all times, pulls the new image and creates two new pods automatically.

For our databases and object storage, failover is configured however the provider sets it. In the case of cloudnativePG, if the master replica fails, a new one is elected automatically from the remaining pods, while a new replica is recovered. On the other hand, Elasticsearch runs 1 master node that is strictly for managing the data pods. If the master node fails, a new master node is created; when a data node fails, a new data node is created. The self healing of the databases will be demonstrated in the next section.

Persistence Design + Verification



Longhorn architecture (from longhorn.io)

For persistence, we are using Longhorn. We have configured Longhorn with the following:


```

apiVersion: storage.k8s.io/v1
kind: StorageClass
metadata:
  name: longhorn-elasticsearch
provisioner: driver.longhorn.io
allowVolumeExpansion: true
reclaimPolicy: Retain # Keep data even if PVC is deleted
volumeBindingMode: Immediate
parameters:
  numberOfReplicas: "2" # Longhorn will replicate data across 2 nodes
  staleReplicaTimeout: "2880" # 48 hours
  fromBackup: ""
  fsType: "ext4"
  dataLocality: "disabled" # Allow scheduling on any node

```

We have set Longhorn to replicate across 2 nodes. We ran our kubernetes cluster within a 3 node setup with 2 worker nodes and 1 master. Setting this option to 3 caused issues as longhorn could not replicate into the master node, and storage was stuck on pending as a result. If we ran more nodes, we could set this higher.

Datalocality was also disabled to allow for even further redundancy, In the event that a node went down, our storage would still persist and be usable on a different node.

We can test persistence by deleting one pod of our elastic cluster, but this is more like testing the failover mechanism.

```

Error from server (NotFound): pods "test-cluster-es-data-1" not found
[stiran20@node-0:~/MyPlace/Manifests/elasticsearch$ kubectl delete pod test-cluster-es-data-1 -n elastic-system
pod "test-cluster-es-data-1" deleted
[stiran20@node-0:~/MyPlace/Manifests/elasticsearch$ kubectl get pods -n elastic-system
NAME                                READY   STATUS    RESTARTS   AGE
elastic-operator-0                  1/1     Running   0           29m
production-kibana-kb-6cfd4cc578-z9cjb 1/1     Running   0           11m
test-cluster-es-data-0              1/1     Running   0           29m
test-cluster-es-data-1              0/1     Init:0/2   0           10s
test-cluster-es-master-0            1/1     Running   0           29m
[stiran20@node-0:~/MyPlace/Manifests/elasticsearch$ ]

```

Deleting a pod

```

"traitVector": null
},
{
  "id": 257,
  "profileId": "105f8431-05b8-4fad-92b8-a0147462b608",
  "bio": "Chris From Tinder. Cleanliness: 7, Personality: 1, HoursAwake: 1",
  "cleanliness": 7,
  "personality": 1,
  "hoursAwake": 1,
  "gender": "Male",
  "traitVector": null
},
{
  "id": 258,
  "profileId": "856363b4-ea6b-45ad-a81f-19185db7c062",
  "bio": "Chris From Tinder. Cleanliness: 5, Personality: 4, HoursAwake: 6",
  "cleanliness": 5,
  "personality": 4,
  "hoursAwake": 6,
  "gender": "Male",
  "traitVector": null
},
{
  "id": 259,
  "profileId": "aa9d12a2-d30b-494e-9966-d10bebbc1055",

```

Endpoint still returns data

To truly test persistence, all of the database cluster's/object storage cluster's pods need to be deleted at the same time, as they invoke the failover mechanisms to ensure high availability. This event is unlikely since the pods are spread across multiple nodes. We believe a critical natural disaster event affecting cloudlab is the only reason this could happen naturally.

```

stran20@node-0:~/MyPlace/Manifests/elasticsearch$ kubectl get pods -n postgres-system
NAME                READY   STATUS    RESTARTS   AGE
postgres-cluster-1  1/1     Running   0           26m
postgres-cluster-2  1/1     Running   0           24m
postgres-cluster-3  1/1     Running   0           23m
stran20@node-0:~/MyPlace/Manifests/elasticsearch$ kubectl delete pods postgres-cluster-1 postgres-cluster-2 postgres-cluster-3 -n postgres-system
pod "postgres-cluster-1" deleted
pod "postgres-cluster-2" deleted
pod "postgres-cluster-3" deleted

```

Deleting ALL the postgres pods (simulating a worst case scenario)

```

{
  "firstName": "Chris",
  "lastName": "From Tinder",
  "attributes": {
    "id": 255,
    "profileId": "010fd5bb-5caf-46ce-a32c-eaba243ff6b4",
    "bio": "Chris From Tinder. Cleanliness: 7, Personality: 6, HoursAwake: 1",
    "cleanliness": 7,
    "personality": 6,
    "hoursAwake": 1,
    "gender": "Male",
    "traitVector": [
      7,
      6,
      1
    ]
  },
  "pictures": [
    {
      "id": 255,
      "fileName": "chris_tinder.png",
      "profileId": "010fd5bb-5caf-46ce-a32c-eaba243ff6b4"
    }
  ],
  "externalMetadataRequests": []
}

```

Data Persists! Thanks Longhorn!

Known Limitations

- Updating the tables in postgres will likely require a full redeployment of the pods, regardless of whether or not that service interacts with the updated table. We haven't tested this yet, but this is likely to be a tedious process to complete.
- Continually getting recommendations is also an area we'd like to improve. If the user decides to match with all their recommendations, the only way to receive new recommendations is via match requests. This occurs because the Elasticsearch recommendations do not change based on the user actions. The API will almost certainly retrieve the same profiles from elasticsearch every time, and the user, who's matched with all of them, will have all of the recommendations removed. To overcome this requires remodelling our elasticsearch models and rewriting queries with user state in mind. However, elasticsearch writes are rather slow, and our app is write-heavy. GraphQL might be an interesting direction to take, but regardless a fundamental redesign of the recommendation system is needed.